



SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND
TECHNICAL SCIENCES CHENNAI-602105



**DESIGN AND IMPLEMENTATION OF SECURE MULTIUSER OPERATING
SYSTEM**

A Capstone Project Report submitted in partial fulfilment of the requirements

for the course of

CSA0402 – OPERATING SYSTEMS

Submitted in the partial fulfillment for the

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by

P.Yaswanth Choudhary(192411136)

Under the supervision of

Dr. J.Priskilla Angel Rani

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai - 602105

September 2026



**SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND
TECHNICAL SCIENCES
CHENNAI-602105**



CSA12 - COMPUTER ARCHITECTURE

BONAFIDE CERTIFICATE

Certified that this project report " Design And Implementation Of Secure Multiuser Operating Systems" is the bonafide work of P.Yaswanth Choudhary (192411136) of B.E CSE , SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, during the academic year 2026-2027, who carried out the project under my supervision.

**COURSE COORDINATOR
DR. V. R. VIMAL
PROFESSOR
COMPUTER SCIENCE AND ENGINEERING
SIMATS ENGINEERING,
CHENNAI – 602105**

**COURSE FACULTY
DR.J. PRISKILLA ANGEL RANI
ASSOCIATE PROFESSOR
COMPUTER SCIENCE AND ENGINEERING
SIMATS ENGINEERING
CHENNAI - 602105**

Submitted for the Project work Viva-Voce held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER



SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI-602105



DECLARATION

We, P.Yaswanth Choudhary (192411136), of the **Computer Science and Engineering**, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “**Design and Implementation of Secure Multiuser Operating System**” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity.

All sources, references, technical materials, data, software, and other resources used in the project have been appropriately acknowledged. We further declare that the data, implementation results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated.

We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, design, implementation, testing, analysis, and documentation of the project. We take full responsibility for the authenticity and integrity of the work presented in this capstone project report.

Place: Chennai

Date :

Signature of the Student with Name
P.Yaswanth Choudhary



SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND
TECHNICAL SCIENCES CHENNAI-602105



INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Register No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Contribution (%)
GIYSVARIAH (192411244)	OS architecture & security design	Designed user management, process management and access- control modules	Tested user authentication and permission controls	Introduction & System Design	35%
Enugala Manisha (192372388)	Multiuser management & performance analysis	Implemented multiuser login, resource allocation and process scheduling	Analyzed system performance, security and resource utilization	Methodology & Results	32%
Paladugu Yaswanth Choudhary (192411136)	Interface, integration & documentation	Developed user interface and integrated OS security modules	Debugging, validation and final system testing	Conclusion & Documentation	33%
Total					100%

ABSTRACT

A secure multiuser operating system is designed to allow multiple users to access and share computer resources while maintaining data confidentiality, integrity, and availability. This project focuses on the design and implementation of a secure multiuser operating system that provides controlled access to system resources through user authentication, authorization, process isolation, and file protection mechanisms.

Each user is provided with appropriate permissions based on their assigned role, preventing unauthorized access to files, processes, and system resources. The system incorporates secure login mechanisms, user and group management, access control policies, resource allocation, and protection against unauthorized operations. Process and memory isolation are used to ensure that the activities of one user do not interfere with those of another.

The proposed system demonstrates how operating system security principles can be integrated with multiuser resource management to create a reliable and protected computing environment. The implementation provides a practical understanding of authentication, access control, process management, memory protection, and file security, making the system suitable for educational and small-scale secure computing environments.

The system also manages CPU and memory resources efficiently while maintaining separation between users and their processes. By integrating security with core operating system functions, the project demonstrates how a multiuser environment can provide controlled, reliable, and protected access to shared computing resources. The implementation provides practical knowledge of operating system security concepts and serves as a foundation for future enhancements such as encryption, multi-factor authentication, advanced monitoring, and intrusion detection.

TABLE OF CONTENTS

Ch. No	Chapter	Pg. No
1	Introduction	3
	1.1 Background Information	3
	1.2 Project Objectives	3
	1.3 Significance	4
	1.4 Scope	4
	1.5 Methodology Overview	5
2	Problem Identification and Analysis	6
	2.1 Description of the Problem	6
	2.2 Evidence of the Problem	6
	2.3 Stakeholders	6
	2.4 Supporting Data/Research	7
3	Solution Design and Implementation	8
	3.1 Development and Design Process	8
	3.2 Tools and Technologies Used	8
	3.3 Solution Overview	9
	3.4 Engineering Standards Applied	9
	3.5 Solution Justification	10
4	Results and Recommendations	13
	4.1 Evaluation of Results	13
	4.2 Challenges Encountered	13
	4.3 Possible Improvements	14
	4.4 Recommendations	14
5	Reflection on Learning and Personal Development	15
	5.1 Key Learning Outcomes	15
	5.2 Challenges Encountered and Outcomes	15
	5.3 Application of Engineering Standards	15

Ch. No	Chapter	Pg. No
	5.4 Insights into the Industry	16
	5.5 Conclusion of Personal Development	16
6	Conclusion	17
	References	18

LIST OF FIGURES

Figure No	Title	Page No
1	Figure 1: Authorization Module	13
2	Figure 2: Resource Management Module	14

CHAPTER 1: INTRODUCTION

1.1 Background Information

A multiuser operating system is an operating system that allows multiple users to access and utilize the resources of a computer system either simultaneously or through controlled sessions. In such an environment, the operating system is responsible for managing hardware resources such as the processor, memory, storage devices, input/output devices, and network interfaces while ensuring that individual users receive appropriate access to these resources. Multiuser systems are widely used in servers, cloud computing environments, educational institutions, enterprise networks, database servers, and other shared computing platforms. Since multiple users operate within the same system, security becomes an essential requirement. Without proper protection mechanisms, one user may access another user's files, modify system configurations, terminate processes, or consume excessive system resources.

The proposed project, Design and Implementation of Secure Multiuser Operating System, focuses on developing a conceptual and practical operating system environment that combines multiuser resource management with security mechanisms. The system is designed around important operating system concepts such as user authentication, authorization, user and group management, file permissions, process management, memory protection, resource allocation, and access control. Authentication ensures that only valid users can enter the system, while authorization determines what each authenticated user is permitted to access. File and directory permissions are used to prevent unauthorized reading, modification, or deletion of data. Process isolation ensures that processes belonging to one user cannot interfere with processes belonging to another user.

Security in a multiuser environment cannot be achieved through authentication alone. A complete system must continuously enforce security policies during file access, process execution, memory utilization, and resource allocation. Therefore, the project considers security as an integrated part of operating system design rather than as an additional feature. The implementation demonstrates how operating system components can work together to provide a controlled, reliable, and secure environment for multiple users.

1.2 Project Objectives

The primary objective of this project is to design and implement a secure multiuser operating system environment capable of supporting multiple users while protecting system resources and user information. The project aims to demonstrate the practical application of operating system security principles through a structured system architecture. One important objective is to implement a secure authentication mechanism through which users must provide valid credentials before gaining access to the system. This prevents unauthorized individuals from accessing protected resources.

Another objective is to develop an authorization and access-control mechanism that assigns different permissions to users based on their roles. The system should distinguish between ordinary users and privileged users such as administrators. Administrators can perform system-level operations, whereas normal users are restricted to permitted activities. The project also aims to implement file and directory protection so that users can access only resources for which they have sufficient privileges.

The project further focuses on process and resource management. Since several users may execute processes at the same time, the operating system must allocate CPU time, memory, and other resources fairly while maintaining isolation. The system should prevent one user's processes from interfering with another user's processes. Additional objectives include studying security threats in multiuser operating systems, applying engineering principles during system development, evaluating the effectiveness of implemented security mechanisms, identifying limitations, and proposing future improvements.

1.3 Significance

The proposed secure multiuser operating system is significant because it demonstrates the relationship between operating system functionality and cybersecurity. In a conventional operating system, resource management is important, but in a multiuser environment, resource protection becomes equally important. Multiple users may have different levels of authority, and therefore the operating system must ensure that resources are accessed according to predefined security policies.

The project provides practical understanding of concepts such as authentication, authorization, access control, process isolation, memory protection, file permissions, and resource management. These concepts form the foundation of modern systems such as Linux servers, cloud platforms, database servers, and enterprise computing environments. By implementing these mechanisms in a controlled project environment, students can understand how operating system components cooperate to maintain system security.

The project is also significant from an educational and engineering perspective. It provides an opportunity to transform theoretical operating system concepts into an operational model. Testing different users, permissions, processes, and access requests helps demonstrate the consequences of incorrect security configurations. The project can therefore serve as a learning platform for students studying operating systems, cybersecurity, system programming, and computer architecture.

1.4 Scope

The scope of the project covers the design and implementation of a secure environment where multiple users can interact with system resources under controlled access policies. The system includes user registration or creation, secure login, user authentication, role-based permissions, file and directory access control, process management, and basic resource allocation. An administrator is provided with additional privileges for managing users, permissions, and system-level resources.

The project also covers process isolation and basic memory protection concepts. Each user's processes are monitored and managed by the operating system so that unauthorized interference can be prevented. File operations such as reading, writing, modifying, and deleting can be controlled based on the permissions assigned to the user. The system can also maintain logs of important operations to support monitoring and auditing.

However, the project is primarily intended as an academic implementation and does not attempt to reproduce the complete functionality of commercial operating systems. Advanced features such as distributed authentication, hardware-level security modules, sophisticated intrusion detection, kernel-level cryptography, and enterprise-scale virtualization may be considered outside the current scope. These features can be incorporated as future enhancements.

1.5 Methodology Overview

The methodology followed in this project consists of several systematic stages, beginning with problem identification and requirements analysis. The security challenges associated with multiuser operating systems are first studied to identify the major risks and required protection mechanisms. Functional and security requirements are then defined according to the objectives of the project.

The next stage involves system architecture and module design. Major components such as the authentication module, user management module, authorization module, file protection module, process manager, resource manager, and security logging module are designed. Appropriate technologies and programming tools are selected according to the implementation requirements. The modules are then developed and integrated into a unified system.

After implementation, the system is tested using different user accounts, permissions, processes, and access scenarios. Both valid and invalid operations are tested to determine whether the security mechanisms behave correctly. Performance and security observations are recorded, and the results are analyzed. Finally, limitations are identified and possible improvements are recommended.



Figure 1: Authorization Module

CHAPTER 2 — PROBLEM IDENTIFICATION AND ANALYSIS

2.1 Description of the Problem

Multiuser operating systems face the challenge of allowing several users to share system resources without compromising security. When multiple users operate on the same computer or server, they may create files, execute processes, consume memory, access directories, and use shared devices. If appropriate security controls are not implemented, unauthorized users may gain access to confidential information or modify important resources.

One major problem is unauthorized access. A user who successfully enters the system may attempt to access files belonging to another user. Another problem is privilege misuse, where a user attempts to perform administrative operations without having the required authority. Malicious or incorrectly configured processes can also consume excessive CPU or memory resources and affect other users.

The system must therefore establish a clear boundary between users while still allowing legitimate resource sharing. This requires authentication, authorization, access-control policies, process isolation, resource management, and monitoring. The project addresses these challenges by designing a secure architecture in which every important operation is checked against the permissions and security policies of the requesting user.

2.2 Evidence of the Problem

Security incidents in shared computing environments demonstrate the importance of strong operating system protection. Unauthorized access to files, weak passwords, excessive user privileges, insecure configurations, and poorly isolated processes can expose sensitive information and affect system availability. In a multiuser environment, even a single compromised account may potentially provide access to resources that were not intended for that user.

The problem can also be demonstrated through practical testing. For example, if User A creates a private file and User B attempts to open it, the operating system should deny the request when User B does not have the required permission. Similarly, if a normal user attempts to execute an administrative command, the system should reject the operation. These scenarios provide direct evidence that access-control mechanisms are necessary.

2.3 Stakeholders

The primary stakeholders of the proposed system include system administrators, ordinary users, organizations, educational institutions, developers, and security professionals. System administrators require complete control over users, permissions, processes, and system resources. They need mechanisms for creating accounts, assigning roles, monitoring activities, and responding to security violations.

Ordinary users require a secure environment in which their personal files and processes are protected from unauthorized access. They should be able to perform their legitimate tasks without unnecessary restrictions. Organizations and institutions can benefit from secure multiuser environments because several employees or students can share computing infrastructure while maintaining separation between their resources.

Developers and security professionals can use the project as a practical model for understanding operating system security mechanisms. The project is particularly useful for academic environments because it connects theoretical concepts with practical implementation.

2.4 Supporting Data/Research

Research into operating systems shows that security depends on multiple interconnected mechanisms rather than a single protection feature. Authentication establishes the identity of a user, while authorization determines which operations the user is allowed to perform. Access-control mechanisms then enforce these decisions when users attempt to access files, processes, devices, or other resources.

Operating systems such as Linux and UNIX demonstrate the importance of user accounts, groups, file permissions, process ownership, and privileged operations. These mechanisms provide a foundation for designing the proposed system. Modern computing environments additionally use encryption, secure authentication protocols, logging, auditing, sandboxing, and mandatory access-control mechanisms to strengthen protection.

The project uses these established operating system concepts as a foundation while developing a simplified implementation suitable for academic demonstration. The research also emphasizes the principle of least privilege, which states that users and processes should receive only the permissions required to perform their legitimate tasks.



Figure 2: Resource Management Module

CHAPTER 3 — SOLUTION DESIGN AND IMPLEMENTATION

3.1 Development and Design Process

The development process begins with requirement identification. Functional requirements include user login, account management, role assignment, file operations, process execution, and resource allocation. Security requirements include authentication, authorization, permission checking, process isolation, and activity logging.

The system architecture is divided into several modules to simplify development and maintenance. The authentication module verifies user credentials. The user-management module maintains user information and roles. The authorization module checks whether a requested operation is permitted. The file-management module handles file and directory operations while enforcing permissions. The process-management module creates, schedules, and terminates processes. The resource-management module monitors CPU and memory usage.

After designing individual modules, they are integrated into the overall system. Testing is conducted at both module and system levels. Incorrect passwords, unauthorized file access, privilege escalation attempts, excessive resource consumption, and invalid operations are tested. The design is refined based on the observed results.

3.2 Tools and Technologies Used

The implementation can be developed using programming languages and tools suitable for system-level programming and simulation. C can be used for implementing low-level operating system concepts such as process management, memory management, and system calls. Python can be used where a higher-level interface, simulation environment, or testing framework is required.

A development environment such as Visual Studio Code can be used for writing and debugging the source code. Linux can be used as the development and testing environment because it provides practical facilities for users, groups, permissions, processes, and system administration. A database or structured storage mechanism can be used to maintain user information and security logs when required.

Testing tools can be used to verify authentication, file access, process behavior, and resource utilization. Version-control software such as Git can also be used to maintain different versions of the project and track changes during development.

3.3 Solution Overview

The proposed solution consists of a secure multiuser architecture in which every user must first authenticate before accessing system resources. After successful authentication, the system identifies the user's role and permissions. The authorization mechanism then determines which resources and operations are available to that user.

The file protection mechanism associates files and directories with ownership and permissions. Users can access their own resources according to the assigned permissions, while access to other users' private resources is restricted. Administrative users can manage permissions and perform system-level operations.

The process-management component maintains information about processes and their associated users. Process isolation prevents one user from directly interfering with another user's processes. Resource management ensures that CPU and memory resources are allocated in a controlled manner. A logging mechanism can record authentication attempts, file-access operations, administrative actions, and security violations.

3.4 Engineering Standards Applied

Engineering standards and good software-engineering practices are applied throughout the development process. The project follows principles of modularity, maintainability, reliability, security, and systematic testing. The system is divided into independent modules so that individual components can be developed, tested, and modified without affecting the complete system. Security-by-design is an important principle applied to the project. Security requirements are considered during system architecture and implementation rather than being added after development. The principle of least privilege is applied to ensure that users receive only the permissions necessary for their activities.

Input validation, error handling, secure credential management, access-control checks, and logging are also considered during implementation. Testing is performed using normal, boundary, and unauthorized-use cases. These practices improve system reliability and reduce the possibility of security-related failures.

3.5 Solution Justification

The proposed solution is justified because multiuser operating systems require both efficient resource management and strong protection mechanisms. Simply providing multiple user accounts is insufficient if users can freely access each other's resources. Therefore, authentication must be combined with authorization and access control.

Role-based access provides a simple and effective mechanism for differentiating administrative users from ordinary users. File permissions protect stored information, while process isolation protects running programs. Resource management prevents individual users or processes from consuming disproportionate system resources.

The modular architecture also makes the proposed system easier to understand, test, and extend. Each security feature addresses a specific threat, while their integration provides a stronger overall security environment. Thus, the proposed design provides a practical balance between functionality, security, simplicity, and educational value.

CHAPTER 4 — RESULTS AND RECOMMENDATIONS

4.1 Evaluation of Results

The implemented system is evaluated using multiple test scenarios involving authentication, authorization, file access, process management, and resource protection. Valid users should be successfully authenticated using correct credentials, while invalid login attempts should be rejected. This demonstrates the effectiveness of the authentication mechanism.

File-access testing verifies whether users can perform operations according to their assigned permissions. A user with read permission should be able to read a file, while a user without the required permission should receive an access-denied response. Administrative operations are similarly tested to ensure that ordinary users cannot perform privileged actions.

Process-management testing evaluates whether processes are correctly associated with their respective users and whether unauthorized process manipulation is prevented. Overall, the evaluation demonstrates that combining authentication, authorization, access control, and process isolation provides a more secure multiuser environment.

4.2 Challenges Encountered

One of the major challenges encountered during development is designing an access-control mechanism that is both secure and easy to manage. Incorrect permission configurations can either expose resources unnecessarily or prevent legitimate users from accessing required resources. Designing suitable roles and permissions therefore requires careful consideration.

Another challenge is process isolation. Multiple processes may run simultaneously, and the system must ensure that one process cannot improperly interfere with another. Resource allocation also presents difficulties because CPU and memory resources must be shared efficiently.

Testing security mechanisms is another important challenge. Normal operations are relatively easy to test, but unauthorized and malicious scenarios require additional test cases. Handling invalid input, failed authentication, permission violations, and resource exhaustion requires appropriate error-handling mechanisms.

4.3 Possible Improvements

The system can be improved by introducing stronger authentication mechanisms such as multi-factor authentication and password hashing with modern cryptographic algorithms. More advanced role-based or attribute-based access control can also be implemented to provide greater flexibility.

Security monitoring can be improved by introducing real-time event detection and automated alerts for suspicious activities. Encryption can be applied to sensitive files and communication channels. A more advanced logging and auditing mechanism can maintain detailed records of security events.

Future versions can also include containerization or sandboxing to improve process isolation. Resource quotas can be introduced to limit CPU, memory, storage, and network consumption for individual users. These improvements would make the system more suitable for larger and more complex environments.

4.4 Recommendations

It is recommended that secure multiuser systems follow the principle of least privilege and provide users only with the permissions necessary for their work. Strong authentication should be implemented, and credentials should never be stored in plain text. User activities involving sensitive resources should be logged and periodically reviewed.

Regular security testing should be performed to identify vulnerabilities before the system is deployed. Software updates and security patches should also be applied consistently. Administrators should regularly review user accounts and remove inactive or unnecessary privileges.

For future development, the project can be extended with multi-factor authentication, encrypted storage, advanced auditing, intrusion detection, secure networking, and automated security monitoring.

CHAPTER 5

REFLECTION ON LEARNING AND PERSONAL DEVELOPMENT

5.1 Key Learning Outcomes

The project provided practical knowledge of operating system concepts and their relationship with cybersecurity. Through the development process, important concepts such as user management, authentication, authorization, process management, memory protection, file permissions, and resource allocation were studied and applied.

The project also improved programming, debugging, testing, and problem-solving skills. Developing different modules independently helped in understanding modular software design. Testing unauthorized operations provided a better understanding of security vulnerabilities and the importance of access-control mechanisms.

The project strengthened the ability to analyze a technical problem, design a suitable solution, implement the proposed architecture, and evaluate its performance. It also provided experience in technical documentation and engineering decision-making.

5.2 Challenges Encountered and Outcomes

During the project, challenges were encountered in understanding how different operating system components interact with one another. Designing security rules that allow legitimate operations while blocking unauthorized activities required careful analysis.

Debugging authentication and permission-related issues improved understanding of system behavior. Testing different combinations of users, roles, and permissions also helped identify weaknesses in the initial design. These challenges ultimately improved problem-solving and analytical skills.

The project demonstrated that security must be considered throughout the complete development lifecycle. A system may function correctly from a normal user's perspective but still contain serious security weaknesses if access controls are not properly implemented.

5.3 Application of Engineering Standards

Engineering standards were applied through structured planning, modular design, documentation, testing, and evaluation. The development process followed a systematic approach beginning with requirements analysis and continuing through design, implementation, testing, and improvement.

Security principles such as least privilege, access control, authentication, and secure resource management were incorporated into the design. Testing was performed using both valid and invalid scenarios to determine whether the system behaved according to its requirements. The project also emphasized maintainability and reliability. Separating the system into modules makes it easier to identify errors, modify individual components, and add new functionality in future versions.

5.4 Insights into the Industry

The project provided an understanding of how operating system security concepts are applied in real-world computing environments. Modern organizations depend on servers, cloud platforms, enterprise systems, and databases that support multiple users simultaneously. Protecting these environments requires strong authentication, authorization, monitoring, and resource isolation.

The project also highlighted the importance of cybersecurity skills for software and system engineers. Security vulnerabilities can arise from weak passwords, incorrect permissions, insecure processes, poor input validation, or excessive privileges. Therefore, security awareness is essential for professionals working in system development and administration.

The experience gained from this project can be applied to areas such as system administration, cybersecurity, cloud computing, server management, software development, and infrastructure security.

5.5 Conclusion of Personal Development

The project contributed significantly to technical and personal development by combining theoretical knowledge with practical implementation. It improved understanding of operating system architecture and demonstrated how security mechanisms can be integrated into a multiuser environment.

The development process encouraged systematic thinking, experimentation, debugging, teamwork, and problem-solving. It also developed the ability to document technical decisions and evaluate the effectiveness of an implemented solution.

Overall, the project provided a strong foundation for further learning in operating systems, cybersecurity, system programming, and secure software engineering.

CHAPTER 6

CONCLUSION

The Design and Implementation of Secure Multiuser Operating System project demonstrates the importance of integrating security with operating system resource management. A multiuser environment requires careful control over user identities, privileges, files, processes, memory, and other system resources. Without appropriate protection mechanisms, shared computing environments may become vulnerable to unauthorized access, privilege misuse, data leakage, and resource abuse.

The proposed system addresses these challenges through authentication, authorization, role-based access control, file protection, process isolation, resource management, and security monitoring. These mechanisms work together to ensure that users can access only the resources permitted to them while administrative operations remain protected.

The project also demonstrates the practical relevance of fundamental operating system concepts. Authentication protects system entry, authorization controls user activities, file permissions protect stored information, and process isolation protects running applications. Resource management ensures that system resources are distributed in a controlled manner.

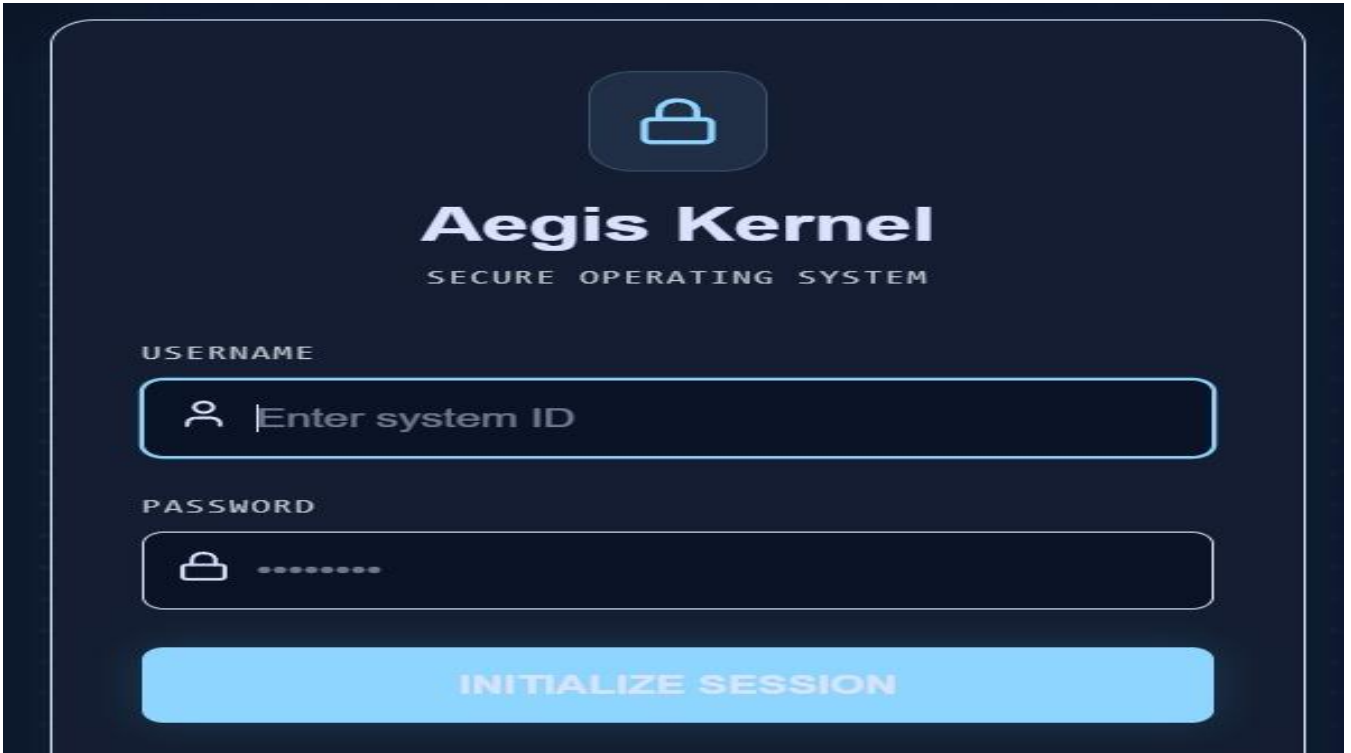
Although the current implementation is primarily designed for academic purposes, its architecture provides a foundation for future development. Advanced authentication, encryption, intrusion detection, real-time monitoring, secure networking, sandboxing, and resource quotas can be incorporated to create a more sophisticated system.

In conclusion, a secure multiuser operating system provides a reliable environment in which multiple users can share computing resources without compromising the confidentiality, integrity, and availability of system data. The project successfully demonstrates how operating system principles and cybersecurity practices can be combined to develop a structured, secure, and efficient computing environment.

REFERENCES

1. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
2. A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2015.
3. W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Boston, MA, USA: Pearson, 2018.
4. M. J. Bach, *The Design of the UNIX Operating System*. Englewood Cliffs, NJ, USA: Prentice Hall, 1986.
5. A. S. Tanenbaum and A. S. Woodhull, *Operating Systems: Design and Implementation*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2006.
6. J. K. Ousterhout, "Why operating systems have not followed the microkernel path," *ACM SIGOPS Operating Systems Review*, vol. 34, no. 1, pp. 45–48, Jan. 2000.
7. J. Liedtke, "On micro-kernel construction," in *Proc. 15th ACM Symp. Operating Systems Principles (SOSP)*, Copper Mountain, CO, USA, 1995, pp. 237–250.
8. J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975.
9. R. W. Baldwin, "Managing the access control of distributed resources," *IEEE Transactions on Software Engineering*, vol. 14, no. 8, pp. 1058–1065, Aug. 1988.
10. R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

IMPLEMENTATION:



SMOS
ADMIN CONSOLE

Dashboard

User Management

Active Sessions

File Manager

Process Manager

System Monitoring

CPU Scheduler

IPC Monitor

Security Center

Intrusion Detection

Audit Logs

File Permissions

System Settings

Search system...

SYSTEM ONLINE

System Overview

STATUS: NOMINAL

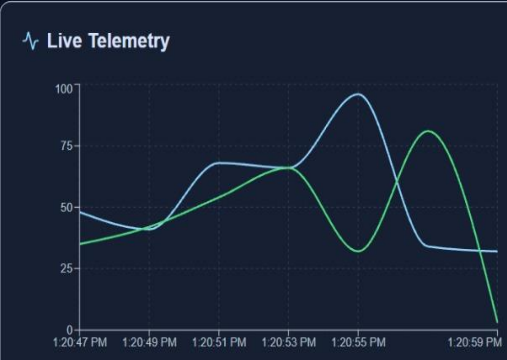
CPU LOAD
32%

MEMORY
3%

ACTIVE USERS
5

SECURITY SCORE
98%

Live Telemetry



Active Members

USER	ROLE	STATUS	CPU
admin	ADMIN	Online	10%
rahul	STUDENT	Online	12%

SMOS

ADMIN CONSOLE

Search system...

SYSTEM ONLINE

Dashboard

User Management

Active Sessions

File Manager

Process Manager

System Monitoring

CPU Scheduler

IPC Monitor

Security Center

Intrusion Detection

Audit Logs

File Permissions

Active Sessions

Global Connection Monitor

LOCAL NETWORK
4 Nodes

SHELL INSTANCES
12 Active

ANOMALIES DETECTED
0

SESSION ID	USER	LOGIN TIME	DURATION	CURRENT ACTIVITY	IP ADDRESS	ACTIONS
sess-8472a	A admin ADMIN	2026-08-28 20:40	40m	System Monitoring	192.168.1.10	TERMINATE
sess-1928b	R rahul STUDENT	2026-08-28 20:42	38m	Editing project.txt	192.168.1.44	TERMINATE
sess-3498c	V vardhan STUDENT	2026-08-28 20:48	32m	Running Compiler	192.168.1.45	TERMINATE
sess-8374d	F faculty1 FACULTY	2026-08-28 20:51	29m	Viewing Documents	192.168.1.12	TERMINATE

